

FPGA Implementation of Morphological Gradient Operation using Xilinx System Generator

Debasish Mukherjee

Computer Science & Engineering

Indian Institute of Information Technology Raichur

Raichur, India

debasish.cse@iiitr.ac.in

Abstract—This paper presents the hardware implementation of morphological gradient algorithm on Field Programmable Gate Array (FPGA) platform using Xilinx System Generator (XSG). Morphological operations are one of the most popular and widely used tools in image processing and analysis due to its flexibility and low design complexity. However, these operations incur additional processing time since they perform repetitive calculations on pixels with varied image dimensions. Thus, hardware implementation is required in order to increase the speed-up and the performance. XSG is a powerful simulink tool that provides hardware specific optimized Very High Speed Integrated Circuit (VHSIC) Hardware Description Language (VHDL)/Verilog code for a design using Xilinx block set. A pipelined architecture is designed by making use of block sets which is synthesized and implemented on Zynq FPGA device. The results of the proposed implementation show significant performance improvement over other existing designs.

Index Terms—Morphological Gradient, XSG, FPGA, Pipelined Architecture

I. INTRODUCTION

Edges in an image are identified by the difference in the intensity of adjacent pixels. Edges represent important information such as size, shape and location of an object [1]. It has been widely used as a pre-processing step in image segmentation [2], feature extraction [3] and object detection [4]. Traditionally, there exist a number of algorithms for edge detection such as Sobel [5], Prewitt [6], Canny [7] and Marr-Hidreth [8]. All these operators have high design complexity and less immune to image noise. Morphological operators (namely dilation and erosion), on the other hand, are less susceptible to noise and provides acceptable performance compared to other edge detection operators. The real-time implementation of morphological operators are challenging since it involve repetitive processing on pixels across varied image dimensions. Thus, a parallel and pipelined architecture is needed that should focus on enhancing the performance and speed-up without increasing the memory requirements.

Designing hardware architecture by programming FPGA is cumbersome and time consuming for novice programmer. Hence, system generator from Matrix Laboratory (MATLAB)

and simulink provides a useful tool for programmers without having the knowledge of hardware descriptive languages, such as VHDL or Verilog. XSG is a graphical environment for modelling an algorithm on to a FPGA hardware platform. It contains Xilinx specific block set that can be used to design a hardware architecture. It generates synthesizable VHDL/Verilog code automatically, thereby eliminating the need for writing several line of code. Modelling a design using XSG is gaining popularity as is evident from the recent research articles that highlights the implementation of image processing algorithms on FPGA platform [9]–[12].

In this paper, the hardware implementation of morphological gradient operation using XSG is proposed. The design is made fully parallel and pipelined, there by exploiting the inter and intra level parallelism among the dilation and erosion operators. Unlike existing architectures, the proposed design directly process pixels on fly, reducing the waiting time for processing individual pixels. This advantages are achieved through lower processing time of the proposed design compared to other existing designs. The rest of the paper is organized as follows: In section 2, the preliminaries on morphological operators are illustrated. The proposed architecture for morphological gradient operator is explained in section 3. Section 4 highlights the experimental results and finally section 5 concludes the paper.

II. PRELIMINARIES

Mathematical morphology are based on set theory where the operations are applied on the pixels of an object by a kernel called the structuring element (SE) [13]. The fundamental morphological operations namely dilation and erosion, consists of finding maximum/minimum in a region defined by the shape and size of the SE. More specifically, dilation (or erosion) finds the maximum (or minimum) within a window which is formed when the centre of SE coincides with the location of each pixel. This process is repeated by sliding the window

over each pixel. Mathematically, dilation is written as shown in equation 1:

$$[I \oplus SE](x, y) = \max[I(x - r, y - s) | (r, s) \in SE] \quad (1)$$

Similarly, erosion is written as shown in equation 2:

$$[I \ominus SE](x, y) = \min[I(x - r, y - s) | (r, s) \in SE] \quad (2)$$

Based on the fundamental operators, more complex or compound morphological operators, such as morphological gradient can be derived. The gradient operator is defined as the difference between dilation and erosion. Mathematically, it is written as shown in equation 3:

$$\text{grad}(I) = [I \oplus SE](x, y) - [I \ominus SE](x, y) \quad (3)$$

III. RELATED WORKS

Hardware implementations of morphological operators are not new and it can be classified broadly into four major groups: (i) Partial Results Reuse (PRR) architecture, (ii) Delay line architecture, (iii) Pipelined architecture and (iv) Systolic array architecture. In PRR, partial results generated during one operation can be reused in the successive operation resulting in reduction in the number of max/min operations. The first PRR architecture was proposed by Sheu et al. [14]. It reduces the hardware cost by reusing the results of the consecutive operations. Chien et al. [15] further reduces the hardware cost by processing using a smaller SEs instead of one large SE.

Delay line architecture stores pixels in line buffer and shift register. The pixels for the current operations are stored in shift register whereas the pixels for the consecutive operations are stored in line buffers. Holzer et al. [16] proposed a delay line architecture for morphological operations. The architecture process pixels in raster scan order and cyclically stores it in a line buffer. Gibson et al. [17] proposed a reconfigurable architecture for morphological operations but the number of delay lines increases with increase in the size of SEs. It can be argued that though delay line architecture has lower complexity but it stores rows of image pixels onto line buffer before processing. This automatically increases the waiting time for processing each pixel resulting in higher computation time.

A pipelined architecture is proposed by Deforges et al. [18] supporting arbitrary convex structuring elements (SEs). The processing is done by decomposing 8-convex SEs into only casual two pixel SEs but often requires deep pipeline for large size SEs. Bartovsky et al. [19] proposed a hardware implementation of morphological filters using two 1D SEs. Though the architecture is able to achieve low latency and memory consumption but is only limited to processing rectangular SEs. Recently, Mukherjee et al. [20] have proposed a parallel and pipelined architecture for morphological filters. The architecture achieved high throughput and frame rate but is limited to processing rectangular SEs.

The systolic array based architectures achieved high performance through concurrent execution of several processing units. Huitzil et al. [21] have proposed a 2D systolic array

architecture for morphological dilation and erosion. The architecture achieved high performance by broadcasting pixels to different processing elements. Though the architecture can process different shapes and size of SEs but the addition of delay elements for synchronization of pixels and SE limit its performance. In [22], a 2D systolic array architecture was proposed. The architecture shows higher processing throughput compared to Huitzil architecture but is only limited to processing 7×7 SE.

Some of the works focused on the efficient implementation of morphological gradient operators by concurrently processing the fundamental dilation and erosion units. Elloumi et al. [23] proposed a flexible architecture for morphological gradient operation through inter-level parallelism of dilation and erosion units. The intra-level parallelism is achieved concurrent processing of pixels belonging to the overlapped region of SE when the pixels are scanned in column-wise fashion. Though the architecture achieved significant speed-up through inter and intra level parallelism but accessing pixels in column-wise order reduces the processing time since it requires pixels to be stored in on-chip buffer before processing. In [24], a delay-line based architecture for morphological gradient operation was proposed detecting edges in four principle directions. Here, the processing time is significantly reduced since it requires all the pixels in the neighbourhood region to be available before the start of any operation.

IV. PROPOSED ARCHITECTURE

This section presents the proposed architecture for morphological gradient operator. To achieve significant performance improvement, the architecture makes use of inter-level parallelism among dilation and erosion units. The proposed pipelined architecture for dilation/erosion process pixels on-fly, thereby eliminating the need for storing pixel onto on-chip buffer before processing. The detailed description of the architectures are illustrated in the following sections:

A. Block Diagrams for Morphological Gradient Operation

Fig. 1 shows the proposed architecture for morphological dilation operation. The same architecture can be deployed for erosion operation by replacing *FIND MAX MODULE1* and *FIND MAX MODULE2* with *FIND MIN MODULE1* and *FIND MIN MODULE2* respectively. As shown in the figure, the architecture consists of (i) *FIND MAX MODULES* (ii) *WRITE SIGNAL MODULE* (iii) *WRITE CONTROL GENERATOR* (iv) *CHIP SELECT MODULE* (v) *RAM ADDRESS GENERATOR MODULES* (vi) *RAM MODULES* and (vii) *DELAY UNITS* respectively. The basic idea is to carry out the processing of one $R \times S$ SE using R number of $1 \times S$ SEs. Initially, pixels from *GateWayIN* passes through the *FIND MAX MODULE1* which finds the maximum value among the pixels in 1×3 ($S = 3$) window of SE (Fig. 2). The results from the *FIND MAX MODULE1* are stored in *RAM MODULES*. The number of memory units used are determined by the row dimension of SE (3 in this case). The data from all the *RAM MODULES* are read and passed into *FIND MAX MODULE2*

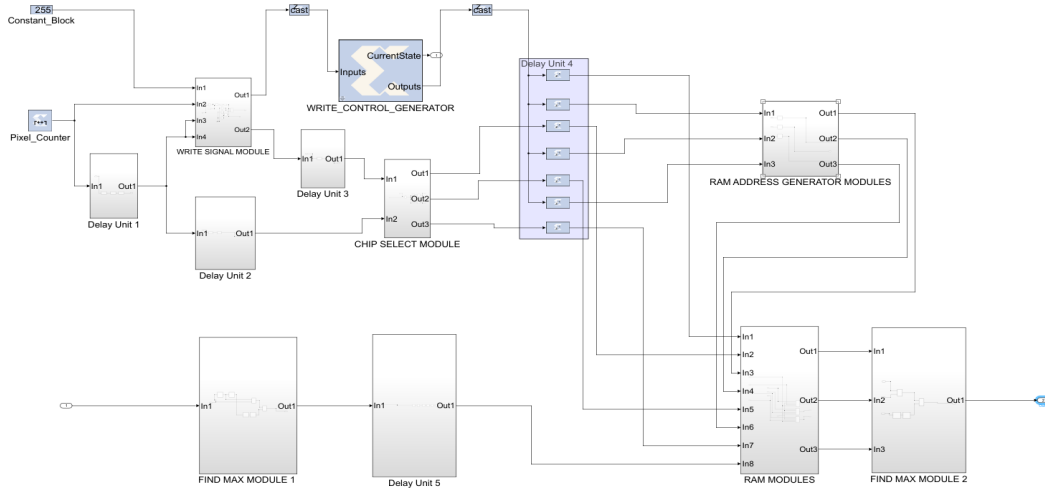


Fig. 1: Block Diagram of 3×3 Dilation Unit

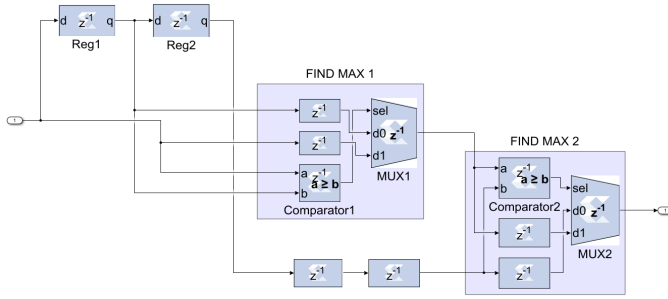


Fig. 2: Block diagram of FIND MAX MODULE1

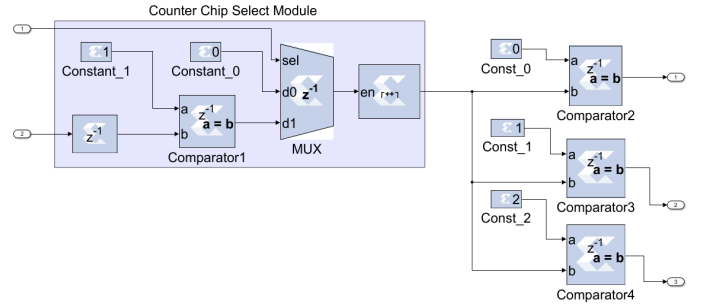


Fig. 4: Block Diagram for CHIP SELECT MODULE

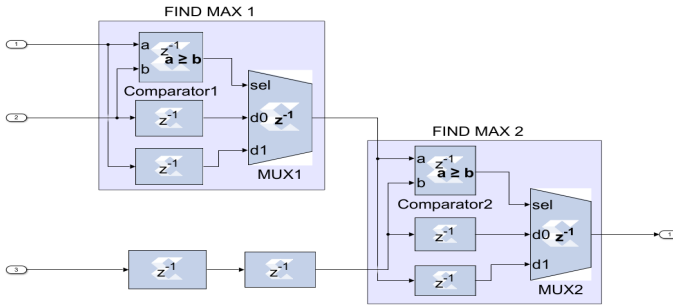


Fig. 3: Block Diagram for FIND MAX MODULE2

for column-wise processing. *FIND MAX MODULE2* (Fig. 3) is also responsible for finding the maximum among R results stored in the RAM modules. The output from this module gives the result of processing one 3×3 SE.

At any given time, among R modules, only one of the RAM module is selected depending on the value of *CHIP SELECT* and *WRITE CONTROL GENERATOR MODULES*. *WRITE CONTROL GENERATOR MODULE* is designed based on two-state mealy machine. Initially, when the input to this module is 0, the machine is in *ZERO* state and the write (wr) is initialized. Once, the output of the first 1×3 SE

is generated the machine goes to *ONE* state and wr signal is asserted. This process continues until there is valid output from the *FIND MAX MODULE1*. During the next clock cycle, the machine goes to state *ZERO* and wr signal is de-asserted. This process is repeated for all the rows of the image. The chip select signal for the *RAM MODULES* are generated from the *CHIP SELECT MODULE* as shown in Fig. 4. As can be seen from the figure, a mod-3 counter is employed which is incremented for each row of SE. A particular RAM module is selected based on the value of the mod counter and the output from the comparator module. Since, at any given time only three rows of data are to be stored, hence, at most three RAM modules are required, thereby minimizing the number of memory consumption in the overall design. The value of the output signal ($out1$ and $out2$) of *WRITE SIGNAL MODULE* is determined by comparing the *pixel_counter* with the number of pipeline stages. *Pixel_counter* module is a Mod-8 counter which is incremented for each pixel of the input image. A number of delay units (pipelined stages) are employed in order to increase the performance by lowering the critical path length of the design.

Fig. 5 shows the block diagram of morphological gradient operator. Here, dilation unit, erosion unit and subtractor mod-

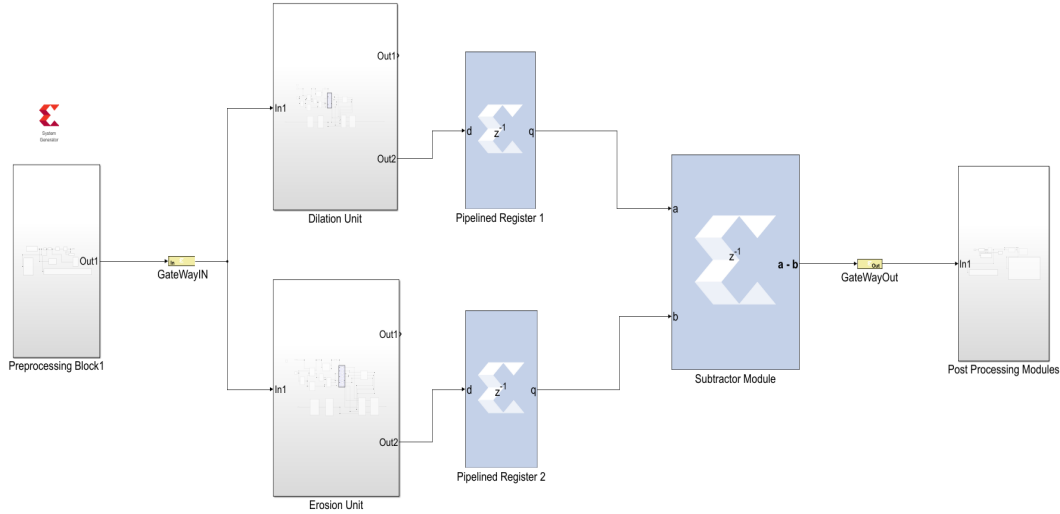


Fig. 5: Block Diagram for morphological gradient operation

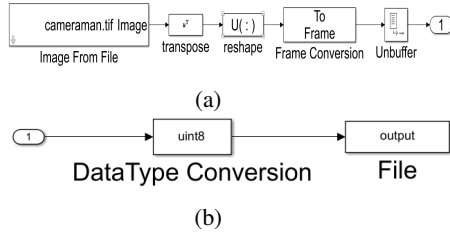


Fig. 6: Block Diagram of (a) pre processing unit and (b) post processing unit

ules are employed. The output from each unit is registered, achieving inter-operator parallelism among the dilation and erosion units. Figure shows the block diagram of pre processing and post processing modules. The detailed description is given in Fig. 6a and Fig. 6b respectively. As shown in Fig. 6a, the input image is read from the file. The image is read as N by M matrix where N is the number of column and M is the number of rows respectively. In order to process it efficiently, this matrix is transposed back to $M \times N$ and is converted to 1D array using reshape module. *To Frame* module converts the output sampling rate to either frame based or sample based. The *unbuffer* unit unbuffers the 2D input into $1 \times N$ output. The 1D output from the morphological gradient architecture is converted back to 2D image using post processing module. As shown in Fig. 6b, the output is passed into *uint8* block which converts the data into 8-bit unsigned integer. The output pixels are then written into a file which is later converted to image using a MATLAB (.m) file.

V. RESULTS AND DISCUSSION

The proposed architectures were synthesized using XSG token and implemented on ZYNQ-7 ZC702 FPGA board. The post implementation reports were obtained to measure the area and timing results.

TABLE I: FPGA resource utilization for 256×256 image size and 3×3 SE

Operation	BRAMs	LUTs	Register
Dilation	1.5/140	97/53200	321/106400
Gradient	3/140	196/53200	645/106400

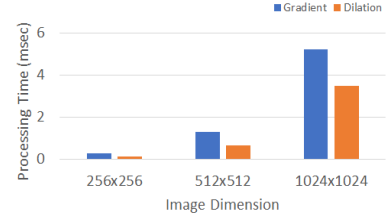


Fig. 7: Processing time of gradient and dilation operator for 3×3 SE

Table I depicts the resource utilization for dilation and gradient operation for fixed image and SE size respectively. Since, the resource consumption of both the dilation and erosion units are similar, hence for the sake of clarity, the results for one of the unit is shown. Though the design can be easily extended to larger SE sizes but the use of higher dimension SEs induces image blurring which limits its use in most of the applications. Table I gives the measure of resource utilization in terms of (i) Block RAMs (BRAMs) (ii) Look Up Tables (LUTs) and (iii) Register. It is worth to point out that the measured resources are close to 1% of that of available resources. This low area utilization makes the architecture more suitable for low cost FPGA design. Since both the dilation and erosion units are concurrently utilized for gradient operation, resulting in twice the resource consumption of gradient unit over the dilation unit.

Fig. 7 shows the processing time, measured in milliseconds



(a) Dilation using 3×3 SE (b) Erosion using 3×3 SE



(c) Gradient using 3×3 SE

Fig. 8: Morphological operation

(msec), for varying image dimensions and fixed SE size respectively. The processing is calculated using the following equation:

$$t_{pt} = M \times N \times t_{clk} + t_{clk} \times l \quad (4)$$

where M and N are row and column dimension respectively, t_{clk} is the clock period and l is the input-to-output latency. It is worth to point out that the architecture can process 190 frames per seconds (fps) for gradient ($t_{pt}=5.24$ msec, frame size= 1024×1024) and 294 fps for dilation operator. It can be noted that measured fps for both the architectures are significantly higher than that of standard real-time processing of 30 frames per second. This makes the architectures more suitable for real-time embedded applications. The reported max clock frequency from the post implementation report for gradient architecture is 230 MHz. This depicts the fact that the use of pipelining has resulted in lower critical path length of the design, thereby increasing the max clock frequency significantly.

Fig. 8 shows the results of dilation, erosion and gradient operation on 256×256 cameraman image using 3×3 SE.

A. Comparative Analysis

For comparison, Table II shows the hardware results of the proposed gradient operator and other existing works. It is worth to point out that the use of different FPGA device in each work would make comparison difficult in terms of resource and timing analysis. Thus, for a fair comparison, the proposed architecture is implemented on similar FPGA device with identical image and SE dimensions. The processing time is calculated based on equation 4 for all the works and is

TABLE II: Comparative Analysis of the proposed gradient architecture for Virtex 6 FPGA. Image size: 512×512 , SE size: 3×3

Method	Register	LUTs	Max Clock Freq. (MHz)	Processing Time (msec)
Kraft et al. [24]	516	689	150	1.8
Elloumi et al. [23]	206	581	356.8	0.78
Proposed Method	645	196	454	0.56

reported in Table II. It is worth to note that the proposed architecture achieved the best solution in terms of max clock frequency and processing time. Though the use of pipelined registers in the design slightly increases the register count but it has significantly lower LUTs consumption compared to other existing works. The architecture proposed by Elloumi et al. [23] performs gradient operation through column-wise access of image data. Such an access to pixels requires data to be stored in on-chip memory column-wise before processing. This will significantly lead to higher waiting time for processing each pixel, thereby reducing the max clock frequency and subsequently increasing the overall processing time. In addition, the LUTs consumption is also higher since it failed to exploit the decomposition property whereby the processing of a 3×3 SE can be replaced by three 1×3 SEs. Kraft et al. [24], on the other hand, is based on delay-line architecture which delays processing of pixels until all the pixels in the current window are available. This increases the processing time and lowers the max clock frequency considerably. The proposed pipelined architecture eliminates the above drawbacks by (i) exploiting the decomposition property and (ii) processing pixels on-fly during on-stream operation. This has resulted in significant improvement in the performance of the proposed architecture over other existing works.

VI. CONCLUSION AND FUTURE SCOPE

This paper presents the hardware architecture of morphological gradient operator using 3×3 square SE. The architecture make use of inter-operator parallelism among dilation and erosion units, thereby improving the max clock frequency of the design. In addition, the input pixels are processed on-fly in raster scan order, lowering the processing time of the overall design. The relatively low resource utilization in terms of register and LUTs count makes the architecture more amenable for low-cost FPGA device. The pipelined architecture for dilation/erosion unit improve the performance by exploiting the decomposition property for processing a 3×3 SE. This has resulted in higher performance through low (about 1% of the available resources) FPGA resource consumption. Though the architecture is limited to processing fixed shape and size of SE but it can be easily extended depending upon the requirement of a specific application. Future scope includes but not limited to (i) processing of different shape and size of SEs, (ii) further improving the performance through block-level parallelism

and (iii) providing an efficient implementation of compound morphological operators.

REFERENCES

- [1] Alan C Bovik, . The essential guide to image processing. Academic Press, 2009.
- [2] Al-Amri, Salem Saleh, N. V. Kalyankar, and S. D. Khamitkar. "Image segmentation by using edge detection." *International journal on computer science and engineering* 2.3 (2010): 804-807.
- [3] Ali, Mohamed, and David Clausi. "Using the Canny edge detector for feature extraction and enhancement of remote sensing images." *IGARSS 2001. Scanning the Present and Resolving the Future. Proceedings. IEEE 2001 International Geoscience and Remote Sensing Symposium (Cat. No. 01CH37217)*. Vol. 5. IEEE, 2001.
- [4] Zhan, Chaohui, et al. "An improved moving object detection algorithm based on frame difference and edge detection." *Fourth international conference on image and graphics (ICIG 2007)*. IEEE, 2007.
- [5] Kanopoulos, Nick, Nagesh Vasanthavada, and Robert L. Baker. "Design of an image edge detection filter using the Sobel operator." *IEEE Journal of solid-state circuits* 23.2 (1988): 358-367.
- [6] Yang, Lei, et al. "An improved Prewitt algorithm for edge detection based on noised image." *2011 4th International congress on image and signal processing*. Vol. 3. IEEE, 2011.
- [7] Canny, John. "A computational approach to edge detection." *IEEE Transactions on pattern analysis and machine intelligence* 6 (1986): 679-698.
- [8] Marr, David, and Ellen Hildreth. "Theory of edge detection." *Proceedings of the Royal Society of London. Series B. Biological Sciences* 207.1167 (1980): 187-217.
- [9] Raut, Neha P., and A. V. Gokhale. "FPGA implementation for image processing algorithms using xilinx system generator." *IOSR Journal of VLSI and Signal Processing (IOSR-JVSP)* 2.4 (2013): 26-36.
- [10] Mohammed, Alareqi, Elgouri Rachid, and Hlou Laamari. "High level FPGA modeling for image processing algorithms using Xilinx system generator." *International Journal of Computer Science and Telecommunications* 5.6 (2014): 1-8.
- [11] Kumar, K. Anil, and M. Vijay Kumar. "Implementation of image processing lab using Xilinx System Generator." *Advances in Image and Video Processing* 2.5 (2014): 27-35.
- [12] Šušteršič, Tijana, et al. "Medical image processing using Xilinx system generator." *International Conference on Computational Bioengineering*. Springer, Cham, 2019.
- [13] Maragos, Petros. "Pattern spectrum and multiscale shape representation." *IEEE Transactions on pattern analysis and machine intelligence* 11.7 (1989): 701-716.
- [14] Sheu, M-H., et al. "A data-reuse architecture for gray-scale morphologic operations." *IEEE Transactions on Circuits and Systems II: Analog and Digital Signal Processing* 39.10 (1992): 753-756.
- [15] Chien, Shao-Yi, Shyh-Yih Ma, and Liang-Gee Chen. "Partial-result-reuse architecture and its design technique for morphological operations with flat structuring elements." *IEEE Transactions on Circuits and Systems for Video Technology* 15.9 (2005): 1156-1169.
- [16] Holzer, Markus, et al. "Optimized hardware architecture of a smart camera with novel cyclic image line storage structures for morphological raster scan image processing." *2012 IEEE International Conference on Emerging Signal Processing Applications*. IEEE, 2012.
- [17] Gibson, Ryan M., et al. "A reconfigurable real-time morphological system for augmented vision." *EURASIP Journal on Advances in Signal Processing* 2013.1 (2013): 1-13.
- [18] Déforges, Olivier, Nicolas Normand, and Marie Babel. "Fast recursive grayscale morphology operators: from the algorithm to the pipeline architecture." *Journal of Real-Time Image Processing* 8.2 (2013): 143-152.
- [19] Bártošský, Jan, et al. "Parallel implementation of sequential morphological filters." *Journal of Real-Time Image Processing* 9.2 (2014): 315-327.
- [20] Mukherjee, Debasish, Susanta Mukhopadhyay, and G. P. Biswas. "FPGA based parallel implementation of morphological filters." *2016 International Conference on Microelectronics, Computing and Communications (MicroCom)*. IEEE, 2016.
- [21] Torres-Huitzil, Cesar. "Fast hardware architecture for grey-level image morphology with flat structuring elements." *IET Image Processing* 8.2 (2014): 112-121.
- [22] Mukherjee, Debasish, Susanta Mukhopadhyay, and G. P. Biswas. "A systolic array architecture for morphological operators using arbitrary structuring elements." *Proceedings of the International Conference on Advances in Information Communication Technology & Computing*. 2016.
- [23] Elloumi, Hejer, et al. "A Highly Flexible Architecture for Morphological Gradient Processing Implemented on FPGA." *Arabian Journal for Science and Engineering* 45.4 (2020): 2675-2684.
- [24] Kraft, Marek, and Andrzej Kasinski. "Morphological edge detection algorithm and its hardware implementation." *Computer Recognition Systems 2*. Springer, Berlin, Heidelberg, 2007. 132-139.